

Web Architecture & Protocols

Week 5 · Foundations module · Textbook Chapter 5

Brian C. Keegan, Ph.D.

Associate Professor, Department of Information Science
University of Colorado Boulder

September 14, 16 & 18, 2026

This week at a glance

This week we examine the layers of protocols below the web. These layers turn a URL into a page on your screen. They also turn web data collection from *magic* into *engineering*.

Monday | Concepts & notebook intro

- Client-server, TCP/IP, DNS, and HTTP as one stack — and how a server decides who, or what, is asking

Wednesday | Notebook lab

- Trace the stack in Python *and* the terminal: a packet capture, a live API call, curl/wget

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 5

Companion notebook

`ch-05-protocols.ipynb`

Bring a laptop

Wednesday is hands-on — we watch the requests, and the packets, fly.

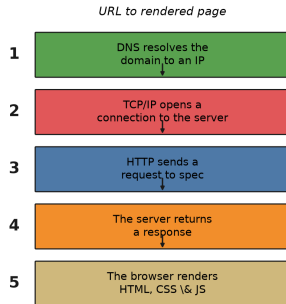
1. Monday • Concepts

Loading a web page, step by step

Type a URL, press Enter, and in milliseconds a precise sequence unfolds:

1. **DNS** resolves the domain name to an IP address
2. **TCP/IP** opens a connection to that server
3. **HTTP** sends a request formatted to spec
4. the server returns a **response**
5. the browser **renders** the HTML, CSS, and JavaScript

Understanding this sequence transforms web data collection **from magic into engineering** — and turns debugging from guesswork into systematic diagnosis.



URL to rendered page, one layer at a time.

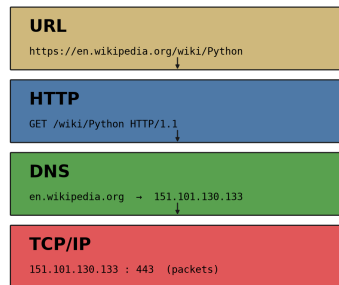
The lower stack: client-server, TCP/IP & DNS

Every scrape and API call is one **request-response** cycle: a **client** (your browser, your Python script) asks a **server** for a resource.

TCP/IP — the transport. Every machine on a public network has an IP address. IPv4 offers $2^{32} \approx 4.3$ billion of them; IPv6 offers $2^{128} \approx 3.4 \times 10^{38}$. Data hops router to router to arrive — as discrete **packets**, each wrapped in its own routing headers.

DNS — the address book. It maps a name like `en.wikipedia.org` to an IP. Names are hierarchical — `subdomain.domain.TLD` — and services follow conventions: `api.agency.gov`, `data.agency.gov`.

Four protocols, one layered conversation



Four protocols, one layered conversation.

Inspecting a page: the Inspector tab

Before writing a line of code, inspect the page by hand. Open dev tools with `Ctrl+Shift+I` or `F12` (`Cmd+Opt+I` on a Mac) — or faster: **right-click the exact thing you care about** and choose **Inspect** (Chrome, Firefox) or **Inspect Element** (Safari). This jumps straight to that element instead of the top of a page that might be thousands of lines long.

- What you see is the **DOM**, not the original HTML — the page *after* JavaScript finished modifying it. View Source shows the original bytes; if an element is in the Inspector but missing from `requests.get(url).text`, JavaScript put it there.
- **Hover** a node to highlight it on the page; the **element picker** icon does the reverse — click the page, land on the node.
- Right-click a node for **Copy selector** / **Copy XPath** — a working starting point for `soup.select()`, though often longer than it needs to be.
- Edits here are local and temporary — safe to delete a cookie banner and see the structure beneath.

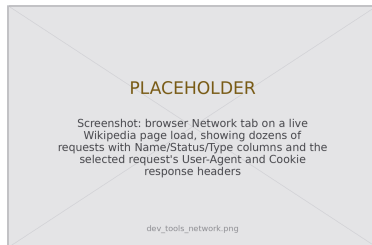
Start on a plain Wikipedia article — commercial sites bury structure under tracking scripts.

Inspecting requests: the Network tab

Same DevTools window, **Network** tab. Reload *after* opening it — it only records what happens while visible. Check **Preserve log** to survive a redirect or multi-step flow.

A page load is dozens or hundreds of requests: HTML, CSS, JS, images, trackers. The **filter bar** narrows it — **Fetch/XHR** is where API calls and dynamically-loaded data live. Each row shows Name, Status, Type, Size, and a **Waterfall** (queuing, DNS, connect, TTFB, download).

Click a request for **Headers** (incl. **User-Agent**, **Cookie**), **Response** (the raw JSON), and **Timing**. Right-click **Copy** → **Copy as cURL** reproduces the exact request — headers, cookies, body.



One page load, dozens of requests.

Most of this chapter's examples end in .org, .gov, or .edu — a U.S.-centered artifact of who built the early internet.

The U.S. has its own country-code TLD, .us — and almost nobody uses it: roughly three-quarters of U.S. sites use .com instead. Most other countries went the opposite way: in Germany, .de is the *default*, unmarked choice for a German site, not a special-purpose alternative the way .us is at home.

Two wrinkles to recognize

Repurposed ccTLDs. .io (British Indian Ocean Territory → “input/output”), .ai (Anguilla → AI startups), .tv (Tuvalu → streaming). Ordinary ccTLDs underneath — you cannot infer origin from the letters.

Non-Latin domains. DNS is ASCII-only; a Cyrillic domain shown in your address bar is transparently **punycode** underneath: xn-80aswg.xn-p1ai. An xn- prefix in requests or urlparse() output is this encoding, not an error.

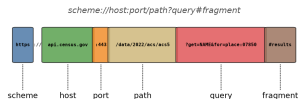
The upper stack: HTTP status codes & URLs

HTTP structures the conversation: a request carries a method (GET, POST, ...) and headers; a response carries a **status code**:

- 200 OK — the body has your data
- 301/302 redirect — follow **Location**
- 403 forbidden — often a missing **User-Agent**
- 404 not found 429 slow down
- 500 the server's problem, not yours

The Network tab's **Status** column already shows you these, before you open a notebook.

URLs identify the target, with a fixed structure:



`scheme://host:port/path?query#fragment`

User-Agent spoofing: identity is self-reported

Every request so far has identified *honestly* — WebDataScience/1.0 (you@colorado.edu). **Spoofing** means sending a **User-Agent** that claims to be a browser or device you are not.

The protocols in this chapter run on an **honor system**: a client just states who it is. Legitimate reasons to change it exist (testing your own site's mobile layout); so do illegitimate ones.

A changed **User-Agent** string alone is weak and easily noticed — not a reliable way to look like a real browser, and modern **Client Hints** headers can expose the mismatch.

Ethical & legal stakes

Not illegal as a technical act — what matters is *what you do it for*. Checking your own site's rendering is unremarkable; defeating a site's stated block is a different act, with the same CFAA/ToS stakes Ch. 2 covers.

Class rule: identify honestly by default; treat a block as a signal to stop, not to route around. Ask your instructor first if a research need suggests otherwise.

2. Wednesday · Notebook Lab

Locating yourself and the server

Your public IP is how the internet sees you; DNS is how a name becomes a routable address:

```
import requests
import dns.resolver

# Your public IP, as servers on the internet see it
print(requests.get("https://api.ipify.org?format=json").json())
# {'ip': '128.138.xxx.xxx'}

# Resolve a domain name to its IP address(es)
for rec in dns.resolver.resolve("en.wikipedia.org", "A"):
    print("IP address:", rec)
```

Before reaching for Python, ping `en.wikipedia.org` from any terminal is the fastest reachability check — no install required, works on every platform.

Getting scapy running: the sudo dance

scapy sends and receives raw packets — a privileged OS operation everywhere, unrelated to scapy itself. Skipping these steps is the #1 way to get stuck:

1. `pip install scapy` # or: `conda install -c conda-forge scapy`
2. Windows only: install Npcap (a driver, not a pip package)
3. Launch Jupyter WITH ELEVATED PRIVILEGES:
 - macOS/Linux: `sudo jupyter notebook`
 - Windows: Anaconda Prompt -> right-click -> Run as administrator

Two gotchas

`sudo` resets your shell's `PATH` — a bare `sudo jupyter notebook` can launch the *wrong* Jupyter and fail with `ModuleNotFoundError`. Fix: `sudo $(which jupyter) notebook`. And: an elevated notebook gives *every* cell full system access, not just the one that needs it — close it when you're done. Machine won't cooperate? The terminal backup on the next frame needs no privileges at all.

Traceroute: reading the hops

```
from scapy.all import traceroute

result, unans = traceroute("en.wikipedia.org", maxttl=20)
```

Each line is one hop — an IP (or hostname) and a round-trip time. Patterns worth recognizing:

- * * * — an unanswered hop, not a broken path; many routers rate-limit or drop probes on purpose
- Private IPs (10., 192.168., 172.16-31.) in the first hop or two — your own network, normal
- A sudden RTT jump — often a long-haul or undersea link, not a problem
- Hostnames often encode geography (iad, ord) — a fast, informal location guess

No privileges needed: `traceroute en.wikipedia.org` (macOS/Linux) or `tracert en.wikipedia.org` (Windows) from any terminal.

Packet sniffing: watching your own traffic

Every message in this chapter travels as **packets**. **Sniffing** captures them off an interface — meaningfully more invasive than anything else here, so this book sniffs only **loopback traffic** you generate yourself, to a server on your own machine:

```
from scapy.all import AsyncSniffer
import requests, threading
from http.server import HTTPServer, SimpleHTTPRequestHandler

httpd = HTTPServer(("127.0.0.1", 8899), SimpleHTTPRequestHandler)
threading.Thread(target=httpd.serve_forever, daemon=True).start()

sniffer = AsyncSniffer(filter="tcp port 8899", iface="lo0") # "lo" on Linux
sniffer.start()
requests.get("http://127.0.0.1:8899/")
packets = sniffer.stop()

for pkt in packets:
    print(pkt.summary())
```

The first packets are TCP's **handshake** — SYN, SYN-ACK, ACK — before a byte of your actual request goes out. Prefer clicking to scripting? **Wireshark** is the standard GUI packet analyzer, same idea, no code.

Diagnosing HTTP errors, not guessing

The status code says *what* failed; the full response says *why*. Read both:

```
def diagnose_request(url, headers=None):
    try:
        r = requests.get(url, headers=headers, timeout=10)
    except requests.exceptions.ConnectionError:
        return print("Connection failed:", url)
    print("Status:", r.status_code)
    print("Content-Type:", r.headers.get("Content-Type", "n/a"))
    print("Bytes:", len(r.content))
    if r.history: # a redirect chain
        print("Redirected from:", r.history[0].url)
    return r
```

A **403** from an API usually means a missing **User-Agent** (a one-line fix); a **403** from a news site may block scraping outright. A **429** means slow down; a **500** is the server's problem — retry later.

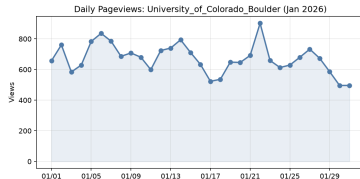
Make an API call, get a time series

The Wikimedia pageviews API rejects the default client with 403 — add a meaningful User-Agent and it returns 200:

```
url = ("https://wikimedia.org/api/rest_v1/"
      "metrics/pageviews/per-article/en.wikipedia/"
      "all-access/all-agents/"
      "University_of_Colorado_Boulder/daily/"
      "20260101/20260131")
ua = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
data = requests.get(url, headers=ua).json()
```

Reshape the nested JSON, then plot it:

```
df = pd.DataFrame(data["items"])
df["timestamp"] = pd.to_datetime(df["timestamp"],
                                format="%Y%m%d00")
df.plot(x="timestamp", y="views")
```



Daily pageviews — spikes line up with real-world events.

User-Agent spoofing, mechanically

Spoofing is just the headers dict you already know — with a value describing a browser you are not:

```
honest = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
spoofed = {"User-Agent": (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36"
)}

# httpbin.org just echoes back whatever header you sent --
# a safe way to see exactly what a server receives
print(requests.get("https://httpbin.org/user-agent", headers=honest).json())
print(requests.get("https://httpbin.org/user-agent", headers=spoofed).json())
```

You already saw this header change real behavior earlier today: the pageviews API's default client got 403; a named one got 200. Same mechanism, different intent.

A loop over one host is your cue for a Session

A bare `requests.get()` opens a fresh connection every time. Looping over many URLs at one host? Create a `Session` — it reuses the connection, remembers headers, and retries automatically:

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
session.headers.update({"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"})

# Retry 429/5xx with backoff of 1s, 2s, 4s; honors Retry-After
retries = Retry(total=3, backoff_factor=1,
                status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

for article in ["Web_scraping", "Data_science"]:
    r = session.get(f"https://en.wikipedia.org/wiki/{article}")
```

Six lines of setup replace the hand-rolled backoff loop — and carry over unchanged into the scraping, Wikipedia, and government-data chapters.

The same requests, from the terminal

Every Python lookup so far has a direct terminal equivalent — no install beyond what your OS already ships:

```
dig en.wikipedia.org # or: nslookup en.wikipedia.org (any OS)
-> same A record as dns.resolver, plus which server answered and its TTL
```

```
whois wikipedia.org
-> a different question than DNS: who registered this, and when
```

```
curl -I https://en.wikipedia.org # headers only -- fast status check
curl -v https://en.wikipedia.org # full request/response, like the Network tab
```

All four take a User-Agent flag (`curl -A "..."`) the same way the `headers` dict does in Python.

Every tool in this section fetches; **neither parses**. The moment you need JSON, HTML, or a DataFrame, you are back to `requests`. They earn their place for a one-off check, a shell script, a server with no Python, or saving straight to disk.

`curl` is built for control (headers, methods, auth); `wget` is built for downloading (saves to disk automatically, resumes, rate-limits). Setting a User-Agent, either way:

```
curl -A "WebDataScience/1.0 (you@colorado.edu)" https://en.wikipedia.org/wiki/Web_scraping
wget --user-agent="WebDataScience/1.0 (you@colorado.edu)" https://en.wikipedia.org/wiki/Web_scraping
```

`curl` prints to your terminal by default (-O saves to a file); `wget` always saves to disk.

Replicating `sleep()`, and fetching many files

`wget` has rate-limiting **built in**; `curl` does not:

```
wget --wait=2 --random-wait URL1 URL2  
-> jitters 0.5x-1.5x of --wait automatically
```

```
for url in URL1 URL2; do curl -O "$url"; sleep 2; done  
-> curl needs a shell loop -- the terminal version of time.sleep()
```

A known list of files, one call, rate-limited for free:

```
wget -i articles.txt --wait=2 --random-wait \  
--user-agent="WebDataScience/1.0 (you@colorado.edu)"
```

Same task as this morning's `Session` loop over three articles — a text file of URLs instead of a Python list.

Parse — and build — URLs with the right tool

Never parse a URL with a regular expression. `urllib.parse` follows the spec:

```
from urllib.parse import urlparse, parse_qs, quote

u = urlparse("https://api.census.gov/data/2022/acs/acs5?get=NAME&for=place:07850")
u.scheme, u.netloc, u.path # 'https', 'api.census.gov', '/data/2022/acs/acs5'
parse_qs(u.query) # {'get': ['NAME'], 'for': ['place:07850']}
```

Two API styles need two builders. **Path-segment** APIs need `quote()` on each component; **query-parameter** APIs take a `params= dict` and let `requests` do the escaping:

```
quote("AC/DC", safe="") # 'AC%2FDC' -- the slash MUST be encoded
requests.get(base, params={"titles": "University of Colorado Boulder"})
```

Mixing them up — query-encoding a value the API expects in its path — is a classic source of confusing 404s.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the chapter:

1. Open the Network tab on a Wikipedia article and on a news site. Count the requests. Give the fraction that is content and the fraction that is tracking.
2. Resolve five related domains with `dns.resolver`. Report the domains that share an IP address.
3. Plot the pageviews of three related articles. Give a possible cause for each spike.
4. Write `diagnose_url()`. Test it on a 200 response, a 404 response, a redirect, an API, and a blocked site.
5. Run the packet-sniffer cell against your own loopback request. Identify the SYN, SYN-ACK, and ACK packets in the output.
6. **5617:** examine the delivery infrastructure of one site. Record the redirect chain, the CDN headers, and the traceroute hops. Write a 500-word memo

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

When a request fails, don't guess.

Ask the stack, in order:

Can I reach the server?

Am I resolving the right address?

What status code came back?

Are my headers correct?

Is my URL well-formed?

3. Friday • Textbook Revisions

Today we make the book better. Each of you opens a **pull request** against `ch-05-protocols.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

A strong PR

- One focused change, clearly titled; explains the problem it fixes
- Builds (`quarto render`) without errors; matches the book's voice
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims; names specifics (line, claim, code)
- Separates “must fix” from “nice to have”; generous, ends with a clear verdict

A revision menu for Chapter 5

High-value targets — pick one and scope it to a single PR:

- **Test the code against live services.** `ipify`, `ip-api.com` (45 req/min), the Wikimedia pageviews API, and DNS results all drift. Run the notebook, report what broke, and fix it.
- **Add a figure.** The chapter still has zero diagrams — a layered-stack or annotated Network-tab figure would anchor the concepts.
- **Deepen “Common Issues to Debug.”** An IPv6 AAAA lookup, and a `curl/wget`-specific gotcha, are still missing.
- **Strengthen an exercise.** Give `diagnose_url()` expected output, a rubric, or a starter cell so it is self-checking.
- **Add a missing citation.** ICANN, Verisign, and `ip-api.com` are all named in the chapter without a link.

Next week: **Parsing static web pages** — turning messy HTML into a clean DataFrame.



A real review comment on a real PR.

Key takeaways

1. The web is a **layered stack**: TCP/IP transports, DNS resolves names, HTTP structures the conversation, URLs identify the target.
2. **Traceroute and packet sniffing** make that transport layer directly observable; the `User-Agent` header — honest or spoofed — is how a server decides who, or what, is asking.
3. Debugging is **systematic diagnosis**, not guesswork: reach the server? right address? which status code? headers correct? URL well-formed?
4. A meaningful `User-Agent` fixes most **403s**; backoff or a `Retry` policy handles **429s**. A `for` loop over one host is your cue for a `Session`.
5. **Never regex a URL**. `urllib.parse` both parses and builds; know path-segment (`quote`) from query-parameter (`params=`) APIs.
6. `requests` has a terminal equivalent in `curl` and `wget` — know when opening a notebook is more overhead than the task deserves.